CSE411 Assignment #4: Bottom-Up Parsers

Sungho Kim (20241065)

October 19, 2026

1 Preliminaries

	Production
	$E' \to E\ \$$ (augmented)
(0)	$E \to E + T$
(1)	$E \to T$
(2)	$T \to T * F$
(3)	$T \to F$
(4)	$F \to \texttt{INT}$
(5)	$F \to (E)$

X	FIRST(X)	FOLLOW(X)
E	$\{\texttt{INT}, (\}$	$\{+,), \$\}$
T	$\{\texttt{INT}, (\}$	$\{+, *,), \$\}$
F	$\{\texttt{INT}, (\}$	$\{+, *,), \$\}$

Table 1: Numbered productions (left) and FIRST/FOLLOW sets (right). As in the lecture, the augmented production ends with the end marker \$.

In every table, sj means *shift* the next input terminal and go to state j. rp means *reduce* by production (p), $X \to \alpha$: pop α off the stack, then go to the state given by GOTO on X. acc means *accept*. Blank cells are errors.

2 The SLR(1), LR(1), LALR(1) Parsing Tables for *valid-grammar.json*

2.1 The SLR(1) Parsing Table for *valid-grammar.json*

	ACTION						GOTO		
State	INT	+	*	(	)	$	E	T	F
0	s1			s2			3	4	5
1		r4	r4		r4	r4			
2	s1			s2			6	4	5
3		s7				acc			
4		r1	s8		r1	r1			
5		r3	r3		r3	r3			
6		s7			s9				
7	s1			s2				10	5
8	s1			s2					11
9		r5	r5		r5	r5			
10		r0	s8		r0	r0			
11		r2	r2		r2	r2			

Table 2: SLR(1) parsing table (12 states, conflict-free).

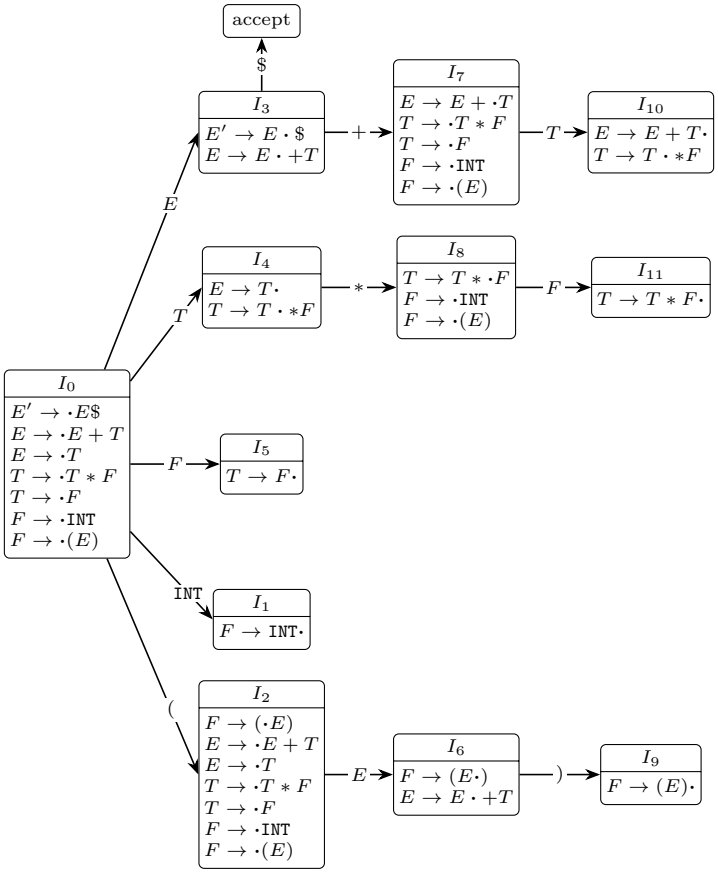


Figure 1: LR(0) automaton used by the SLR(1) table. Only the arrow that first reaches each state is drawn. The other transitions, such as $I_7 \xrightarrow{\texttt{INT}} I_1$, are the remaining s$j$ and GOTO entries of Table 2. A completed item $X \to \alpha\cdot$ (dot at the end) in I_k means that α can be reduced to X. Hence, if $X \to \alpha$ is production (p), row k gets rp for every terminal in FOLLOW(X); e.g., $F \to$ INT$\cdot$ in I_1 gives r4 on $+, *,), \$$.

2.2 The LR(1) Parsing Table for *valid-grammar.json*

	ACTION						GOTO		
State	`INT`	+	*	(	)	$	*E*	*T*	*F*
0	s1			s2			3	4	5
1		r4	r4			r4			
2	s6			s7			8	9	10
3		s11				acc			
4		r1	s12			r1			
5		r3	r3			r3			
6		r4	r4		r4				
7	s6			s7			13	9	10
8		s14			s15				
9		r1	s16		r1				
10		r3	r3		r3				
11	s1			s2				17	5
12	s1			s2					18
13		s14			s19				
14	s6			s7				20	10
15		r5	r5			r5			
16	s6			s7					21
17		r0	s12			r0			
18		r2	r2			r2			
19		r5	r5		r5				
20		r0	s16		r0				
21		r2	r2		r2				

Table 3: Canonical LR(1) parsing table (22 states, conflict-free).

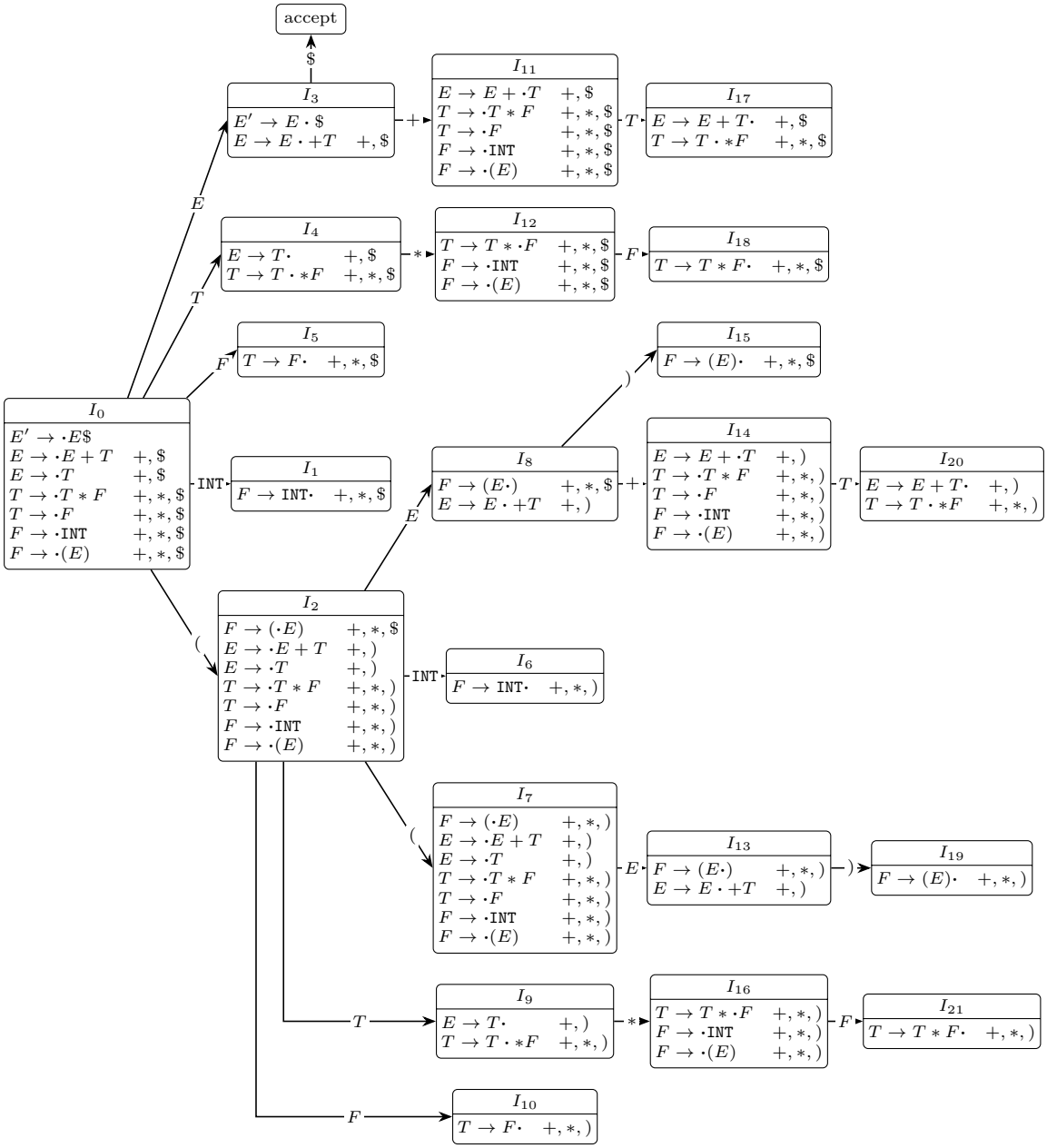


Figure 2: LR(1) automaton. Each line is a production followed by its lookahead terminals, separated by commas; e.g., $F \to \texttt{INT}\cdot \quad +, *, \$$ in I_1. $E' \to \cdot E\ \$$ needs no lookahead, because the \$ after E already plays that role. The upper states (lookaheads with \$) belong to the outermost expression. The lower states (lookaheads with)) belong to an expression inside parentheses. A completed item $X \to \alpha \cdot \quad z_1, \ldots, z_n$ in I_k gives rp in row k *only* for $z_1, \ldots, z_n$. As in Figure 1, only the arrow that first reaches each state is drawn.

2.3 The LALR(1) Parsing Table for *valid-grammar.json*

	ACTION						GOTO		
State	`INT`	+	*	(	)	$	*E*	*T*	*F*
0	s1			s2			3	4	5
1		r4	r4		r4	r4			
2	s1			s2			6	4	5
3		s7				acc			
4		r1	s8		r1	r1			
5		r3	r3		r3	r3			
6		s7			s9				
7	s1			s2				10	5
8	s1			s2					11
9		r5	r5		r5	r5			
10		r0	s8		r0	r0			
11		r2	r2		r2	r2			

Table 4: LALR(1) parsing table (12 states, conflict-free). For this grammar it is identical to the SLR(1) table (Table 2).

LALR(1) state	0	1	2	3	4	5	6	7	8	9	10	11
Merged LR(1) states	0	1, 6	2, 7	3	4, 9	5, 10	8, 13	11, 14	12, 16	15, 19	17, 20	18, 21

Table 5: Each LALR(1) state merges the LR(1) states (Figure 2) that contain the same items when lookaheads are ignored.

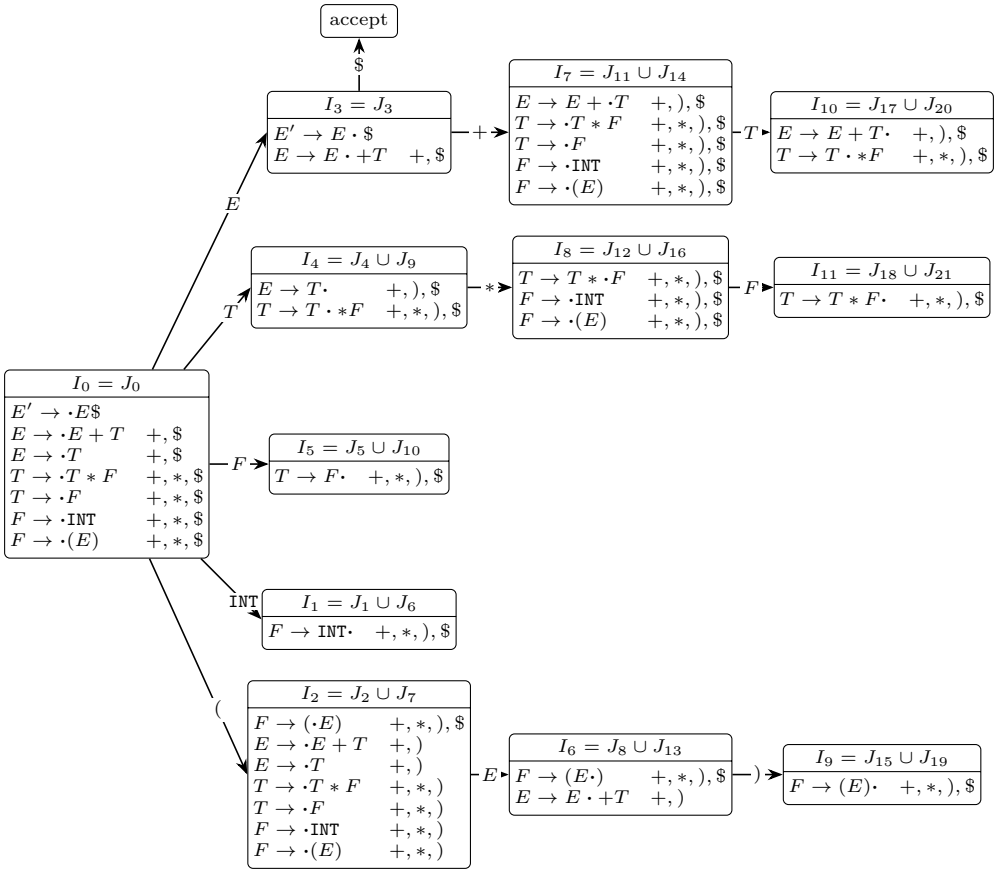


Figure 3: LALR(1) automaton. J_k is state k of the LR(1) automaton (Figure 2), and each lookahead list is the union over the merged states. Without the lookaheads, the states and arrows are exactly those of the LR(0) automaton (Figure 1).

3 Parsing Table Construction

Each construction builds an automaton of item sets, then uses its transitions and completed items to fill the parsing table. An edge $I_k \xrightarrow{Y} I_j$ gives sj in ACTION$[k, Y]$ if Y is a terminal, and j in GOTO$[k, Y]$ if Y is a nonterminal. The edge $I_3 \xrightarrow{\$}$ accept gives acc. States are numbered in the order they are added: the implementation scans the states in that order and tries the symbols in the order $\texttt{INT}, +, *, (,), E, T, F$. The methods use different state sets and reduction lookaheads. A cell with two actions would be a conflict. None of the three tables has one, so the grammar is SLR(1), LR(1), and LALR(1).

3.1 SLR(1) Construction

1. Compute FIRST and FOLLOW (Table 1, Algorithm A.1).

2. Build the LR(0) automaton (Figure 1) with Algorithm A.2, starting from CLOSURE($\{E' \to \cdot E \ \$\}$). This yields 12 states.

3. Fill the table (Algorithm A.3). A completed item $X \to \alpha\cdot$ in I_k puts rp in every column $a \in$ FOLLOW(X). For example, $I_1 = \{F \to \texttt{INT}\cdot\}$ reduces by (4) on FOLLOW$(F) = \{+, *,), \$\}$. I_4 contains both $E \to T\cdot$ and $T \to T \cdot *F$, so it reduces by (1) on FOLLOW$(E) = \{+,), \$\}$ and shifts on $*$. Since $* \notin$ FOLLOW(E), there is no shift/reduce conflict. The same applies to I_{10}.

3.2 LR(1) Construction

1. Each item now carries a lookahead z: a terminal that can come next once the item is completed. For an item $X \to \alpha \cdot Y\beta \quad z$, CLOSURE gives each new item $Y \to \cdot\gamma$ the lookaheads FIRST(βz)

(Algorithm B.1). For example, in I_2 the items for E get $+,)$, because E is followed by) in $F \to (\cdot E)$ and by $+$ in $E \to \cdot E + T$.

2. Run the main loop of Algorithm A.2 with this CLOSURE and GOTO. Two states are equal only if their items *and* lookaheads match. So every LR(0) state that can be reached both outside parentheses (followed by \$) and inside them (followed by)) splits in two. Examples are I_1 and I_6 (both $F \to \texttt{INT}\cdot$), and I_5 and I_{10}. Only the states with E' items (I_0 and I_3) occur solely at the top level. The other 10 LR(0) states appear twice, so there are $12 + 10 = 22$ states.

3. Fill the table (Algorithm B.2). A completed item $X \to \alpha \cdot \quad z_1, \ldots, z_n$ reduces *only* on $z_1, \ldots, z_n$. Hence row 1 reduces on $+, *,$ \$ but not on), and row 6 reduces on $+, *,)$ but not on \$. The SLR table instead reduces on all of FOLLOW(F). For example, after reading a top-level `INT` with) next in the input, LR(1) reports an error in state 1; SLR(1) first reduces by (4).

3.3 LALR(1) Construction

1. Build the LR(1) automaton. Then merge all states that contain the same items when lookaheads are ignored, taking the union of their lookaheads (Algorithm C.1). Merged states are numbered in the order in which their first LR(1) state appears. Table 5 lists the 12 merged groups.

2. Merged states have transitions on the same symbols, and those transitions lead to states that are merged together as well. So the merged automaton is still deterministic. Without lookaheads it is the LR(0) automaton, with the same numbering here (Figure 3).

3. Fill the table exactly as for LR(1), using the merged lookaheads (Algorithm B.2). Merging can create reduce/reduce conflicts but never new shift/reduce conflicts. No conflicts occur for this grammar. For every completed item here, the merged lookaheads equal the FOLLOW set of its left-hand side; for example, I_4 has $E \to T \cdot \quad +,), \$$, and FOLLOW(E) $= \{+,), \$\}$. So the LALR(1) table coincides with the SLR(1) table.

Appendices

Appendix A SLR(1) Construction

Algorithm A.1 computes FIRST and FOLLOW until the sets stop changing. FIRST($Y_1 \cdots Y_n$) is FIRST(Y_1) without ε, plus FIRST($Y_2 \cdots Y_n$) if Y_1 can derive ε; the empty string has FIRST = $\{\varepsilon\}$. Algorithm A.2 is the LR(0) automaton construction of the lecture (*LR.html*, slide 23). Algorithm A.3 fills the SLR(1) table from that automaton.

Algorithm A.1 FIRST and FOLLOW sets

FIRST(a) $\leftarrow \{a\}$ for each terminal a; FIRST(X) $\leftarrow \emptyset$ for each nonterminal X
repeat
  for all productions $X \to Y_1 \cdots Y_n$ **do**
    add FIRST($Y_1 \cdots Y_n$) to FIRST(X)
  end for
until no FIRST set changes
FOLLOW(X) $\leftarrow \emptyset$ for each nonterminal X ▷ $E' \to E$ \$ puts \$ in FOLLOW(E)
repeat
  for all productions $X \to \alpha Y \beta$ with Y a nonterminal **do**
    add FIRST(β) $\setminus \{\varepsilon\}$ to FOLLOW(Y)
    if $\varepsilon \in$ FIRST(β) **then** add FOLLOW(X) to FOLLOW(Y)
    end if
  end for
until no FOLLOW set changes

Algorithm A.2 LR(0) automaton construction (lecture slide 23)

$CS \leftarrow \emptyset$ ▷ a set of closures
$CS \leftarrow CS \cup \{\textsc{Closure}(\{E' \to \cdot E\ \$\})\}$ ▷ E is the start nonterminal
$\mathit{Edges} \leftarrow \emptyset$
repeat
  for all $C \in CS$ **do**
    for all items $X \to \alpha \cdot Y\beta$ in C **do** ▷ Y is a terminal or a nonterminal
      if Y is not \$ **then**
        let J be $\textsc{Goto}(C, Y)$
        $CS \leftarrow CS \cup \{J\}$
        $\mathit{Edges} \leftarrow \mathit{Edges} \cup \{C \xrightarrow{Y} J\}$
      else
        $\mathit{Edges} \leftarrow \mathit{Edges} \cup \{C \xrightarrow{\$} \text{accept}\}$
      end if
    end for
  end for
until CS and *Edges* do not change

function $\textsc{Closure}(I)$
  repeat
    for all items $X \to \alpha \cdot Y\beta$ in I **do**
      for all productions $Y \to \gamma$ **do**
        $I \leftarrow I \cup \{Y \to \cdot\gamma\}$
      end for
    end for
  until I does not change
  return I
end function

function $\textsc{Goto}(I, Y)$
  $J \leftarrow \emptyset$
  for all items $X \to \alpha \cdot Y\beta$ in I **do**
    $J \leftarrow J \cup \{X \to \alpha Y \cdot \beta\}$
  end for
  return $\textsc{Closure}(J)$
end function

Algorithm A.3 SLR(1) table construction

Require: LR(0) automaton (CS, Edges) with states $I_0, I_1, \ldots$
for all edges $I_k \xrightarrow{Y} I_j$ in *Edges* **do**
  if Y is a terminal **then** add sj to ACTION$[k, Y]$
  else GOTO$[k, Y] \leftarrow j$
  end if
end for
for all edges $I_k \xrightarrow{\$}$ accept in *Edges* **do** add acc to ACTION$[k, \$]$
end for
for all states I_k and completed items $X \to \alpha\cdot$ in I_k **do**
  let (p) be the number of production $X \to \alpha$
  for all $a \in \text{FOLLOW}(X)$ **do** add rp to ACTION$[k, a]$
  end for
end for
report every ACTION cell with more than one action as a conflict

Appendix B LR(1) Construction

Algorithm B.1 is the LR(1) automaton construction of the lecture (slide 24). Its main loop is that of Algorithm A.2; only Closure and Goto change, so that every item $(X \to \alpha \cdot \beta, z)$ carries a lookahead terminal z. Algorithm B.2 fills the table, reducing only on each item's own lookahead.

Algorithm B.1 LR(1) automaton construction (lecture slide 24)

1: run lines 1–16 of Algorithm A.2 with the Closure and Goto below
▷ $E' \to \cdot E$ \$ needs no lookahead: FIRST(\$ z) = {\$} for every z

2: **function** Closure(I)
3: **repeat** ▷ z: lookahead terminal
4: **for all** items $(X \to \alpha \cdot Y\beta, z)$ in I **do**
5: **for all** productions $Y \to \gamma$ **do** ▷ Y is a nonterminal
6: **for all** $w \in$ FIRST(βz) **do**
7: $I \leftarrow I \cup \{(Y \to \cdot\gamma, w)\}$
8: **end for**
9: **end for**
10: **end for**
11: **until** I does not change
12: **return** I
13: **end function**

14: **function** Goto(I, Y)
15: $J \leftarrow \emptyset$
16: **for all** items $(X \to \alpha \cdot Y\beta, z)$ in I **do**
17: $J \leftarrow J \cup \{(X \to \alpha Y \cdot \beta, z)\}$ ▷ the lookahead is kept
18: **end for**
19: **return** Closure(J)
20: **end function**

Algorithm B.2 LR(1) table construction (also used for LALR(1))

Require: LR(1) automaton $(CS, Edges)$ with states $I_0, I_1, \ldots$
1: fill the sj, GOTO, and acc entries as in lines 1–7 of Algorithm A.3
2: **for all** states I_k and completed items $(X \to \alpha\cdot, z)$ in I_k **do**
3: let (p) be the number of production $X \to \alpha$
4: add rp to ACTION$[k, z]$ ▷ only the item's own lookahead
5: **end for**
6: report every ACTION cell with more than one action as a conflict

Appendix C LALR(1) Construction

Algorithm C.1 merges the LR(1) states that contain the same items once lookaheads are ignored (strip below). Lines 2–8 merge the states, and line 9 redirects every edge to the merged states. The result is still deterministic: merged states have transitions on the same symbols, and those transitions lead to states that are merged together as well. The table is then filled by Algorithm B.2.

Algorithm C.1 LALR(1) by merging LR(1) states

Require: LR(1) automaton with states $J_0, \ldots, J_{n-1}$ and *Edges*; $\text{strip}(J) = \{X \to \alpha \cdot \beta \mid (X \to \alpha \cdot \beta, z) \in J\}$

$m \leftarrow 0$ ▷ number of LALR(1) states so far
for $k \leftarrow 0$ **to** $n-1$ **do** ▷ in LR(1) state order
  if $\text{strip}(J_k) = \text{strip}(I_i)$ for some $i < m$ **then**
    $I_i \leftarrow I_i \cup J_k$; $num(k) \leftarrow i$ ▷ union of lookaheads
  else
    $I_m \leftarrow J_k$; $num(k) \leftarrow m$; $m \leftarrow m+1$
  end if
end for
$Edges' \leftarrow \{I_{num(k)} \xrightarrow{Y} I_{num(j)} \mid J_k \xrightarrow{Y} J_j \in Edges\} \cup \{I_{num(k)} \xrightarrow{\$} \text{accept} \mid J_k \xrightarrow{\$} \text{accept} \in Edges\}$
return states $I_0, \ldots, I_{m-1}$ and $Edges'$